本章路線

這章要回答的核心問題是：分層 Windows 架構如何在 user/kernel 邊界上組織物件、程序、I/O 與服務？

前置知識。 OS 結構、protection domain、file/I/O。

- 辨識 user mode/kernel mode 元件
- 分析 object manager 與 handle 權限
- 分離 7 版本案例與當代實作

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Windows7是NT家族案例，不是所有Windows版本的總稱

本章採原書的Windows7／NT架構脈絡，區分user-mode subsystem／services與kernel-mode executive、kernel、HAL及drivers。不將早期DOS-based Windows、Mach microkernel或現代Windows新增功能回填為相同契約。

名詞	本章意思	不可直接推論
Win32 API	應用使用的程序／物件／I/O等介面	每個API都必然system call
NT kernel/executive	受信任排程、memory、object、I/O等實作	每個component獨立address space
compatibility	既有程式在指定環境可運行	所有架構／bitness／舊subsystem都支援
portability	讓硬體依賴集中且有移植路徑	每個driver binary跨CPU直接可用

歷史圖若出現OS/2、Win16或POSIX subsystem，須按其世代／版本理解，不能因此宣稱每一套Windows7都可執行所有舊binary。後續算例均明列執行、配置及故障模型。

從外部介面到受信任服務的追蹤

例應用ReadFile：library準備參數，受控kernel入口驗證handle與buffer；I/O manager／filesystem／cache與memory manager決定命中或送driver。若等裝置可使thread等待，完成後再依結果回到應用。

一個API名稱不足以決定是否阻塞、是否copy、是否有IRP或幾次context switch；fast/cached路徑可能不同。先記錄handle開啟旗標、檔案／裝置種類、cache狀態及完成模式，再比較路徑。

NT的模組劃分包含大量kernel-mode元件。稱它有layered design，不能推成每次file service都跨user-mode server或自帶獨立failure domain。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

從外部介面到受信任服務的追蹤 （續）

Windows7 / NT：責任模組不等於各有獨立address space



藍=user，綠=kernel-mode模組，灰=HAL硬體抽象；下排是責任視圖，非每條I/O都固定依序穿過。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

設計目標要連到具體強制機制

目標	可用機制	不能被目標名稱保證
security	tokens、security descriptors、handle granted access	kernel/driver沒有漏洞
reliability	保護邊界、reference lifetime、錯誤回報／恢復	一個錯誤不會影響共享kernel
performance	cache、async I/O、thread scheduling	全部工作都低latency
extensibility	object／driver interfaces	任意擴充都安全
portability	HAL、架構專屬層、較高階程式碼	移植零成本
internationalization	Unicode介面與地區資料	字元數等於byte數
energy／dynamic devices	power states、PnP生命期	移除裝置不會遇未完成I/O

HAL隔離interrupt／timer／平台細節的一部分；不是把所有硬體差異、memory model與driver責任消除。支援SMP也不使共享更新自動race-free。

Unicode、相容性與緩衝長度的真單位

Win32 的 W 介面使用 UTF-16 code units，A 介面涉及 code page 轉換；這些單位不能直接等同 UTF-8 bytes 或 Unicode scalar 數。

字串 A、U+1F600、B 編碼為 0041 D83D DE00 0042，共 4 個 16-bit code units、3 個 Unicode scalars、8 bytes 資料；若需終止 NUL，再加 2 bytes，共 10 bytes。4-byte buffer 無法容納完整資料與終止符。

以 wchar_t units 表示的容量不可傳入 byte 數；byte length 也不可直接使用 scalar 數。Unicode API 不保證所有檔名都已正規化成同一等價形式。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

power／PnP的安全性依ownership，而不是只靠通知

固定狀態working→stop-pending→stopped→removed；這是生命期模型，不是完整PnP IRP表。stop-pending先擋新submit，再等待／取消既有requests並確認完成，才能釋放device context、DMA映射及buffers。

若仍有request引用context，先free後interrupt才完成，會use-after-free；若取消已提出但driver還沒終止DMA，也不能重用buffer。相反，device斷線可能使request以錯誤完成，應用必須接受failure而非永等成功。

可攜與節能policy都必須保留這個原則：任何能觸及資源的execution／device引用結束前，不能把該資源交給不相容的新owner。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

kernel、executive、HAL分工

元件	主要責任	典型狀態
kernel dispatcher	thread選取、dispatch、同步／trap基礎	ready queues、thread state
executive managers	object、memory、process、I/O、安全、cache等	handles、PTE/PFN、IRP
HAL	平台／interrupt／timer等硬體介面抽象	硬體路由與控制
drivers	特定device、filter、filesystem等路徑	device objects、queues、buffers
user services／subsystems	登入、背景服務、應用環境等	user address spaces與IPC

executive與kernel layer在此是實作責任名稱，兩者都在kernel mode；不要把同一thread進kernel時所用的kernel stack說成另創一個可獨立排程的「第二thread」。

thread priority和IRQL是兩個不同次序

Windows7 dispatcher選可在該CPU執行的最高priority ready thread；variable1…15、real-time16…31，0為idle特殊用途。variable boost及其decay不低於base、上限不進real-time範圍；具體boost與quantum受條件影響，不把本節q2當全系統固定值。

IRQL控制當前processor可處理哪些interrupt／kernel活動及允許的操作，並非另一個0…31 thread priority。

x86/x64常用IRQL	典型執行	限制
PASSIVE_LEVEL=0	一般thread／worker	仍須按API規則判斷可否等待
APC_LEVEL=1	部分kernel APC活動	有特定APC／呼叫限制
DISPATCH_LEVEL=2	普通DPC、部分spinlock區段	不可作非零時間的blocking wait；不可觸發pageable存取fault
device DIRQL	ISR／device同步	短處理，限制更嚴

在某CPU提高IRQL不會讓另一CPU停止存取共享物件；跨CPU互斥仍需同步。real-time priority也不保證hard deadline。driver規則。

固定priority模型：每次選取可回到queue規則

單CPU、無boost／decay、同priority q2、較高priority立即搶占；被高者搶占後保留本slice餘量且回同級前方。P1/P2 priority8，t0ready，bursts4/3；P3 priority10，t1ready、burst2。

區間	選取原因	結束時狀態
0–1 P1	同級先入	P3較高priority到達；P1餘burst3／slice1
1–3 P3	priority10	P3完成
3–4 P1	同級前方且保留slice1	餘burst2，slice用完回尾
4–6 P2	priority8 queue前方	餘burst1回尾
6–8 P1	同級前方	完成
8–9 P2	唯一ready	完成

completion P1/P2/P3=8/9/3，CPU工作9。此模型只隔離priority與quantum規則；真實系統的boost、quantum與processor條件不同時，須依相應規則另算。

Priority 搶占與 queue 追蹤

單 CPU：priority 搶占與 ready queue

教學模型：P1、P2 priority 8；P3 priority 10；quantum = 2



時間與事件	選擇前 priority 8 ready queue（左側先選）	
t=0	P1 → P2	P1 先入
t=1	P1 → P2	P3 搶占；P1 保留 slice 1
t=3	P1 → P2	P3 完成；P1 接著用完 slice
t=4	P2 → P1	P1 slice 用完，回隊尾
t=6	P1 → P2	P2 尚餘 1，回隊尾

完成時間：P1 = 8、P2 = 9、P3 = 3

固定規則範例；不代表 Windows 7 的實際 quantum 或完整排程行為。

P1 在 t=1 被 P3 搶占，保留 1 單位時間片；t=3 恢復後於 t=4 用完，回到 priority 8 queue 尾端。完成時間為 P1=8、P2=9、P3=3。這是固定教學模型，並非 Windows 7 的實際排程保證。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

dispatcher objects：signaled狀態不是通用訊息queue

manual-reset event一旦signaled，維持到ResetEvent；可讓多個waiters通過。auto-reset event在滿足一個wait後回nonsignaled；若已signaled又SetEvent，不能累加成兩個獨立tokens。因此需計數的resource要用semaphore或自己的queue，而不是拿event當無限計數器。

mutex具有owner，持有者才能依協定release；Wait回WAIT_ABANDONED表示前owner終止等異常，取得者須檢查被保護資料一致性，不能等同資料已修復。mutex並不自動防兩個mutex的lock-order deadlock。

本章用可見semaphore count0…limit，count0的waiter睡眠，release可喚醒並handoff許可；不要套入第5章另一種內部signed counter而混淆外部count。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

ISR、DPC、APC與worker的執行契約

ISR先辨識／確認裝置來源，完成必要最小動作；普通DPC延後一部分處理，但仍在DISPATCH_LEVEL，不能因叫「deferred」就執行可blocking的filesystem I/O。需要sleep/pageable服務的工作送適當PASSIVE_LEVEL worker，並延長其引用的context生命期。

APC排給特定thread；一般user APC通常在該thread進alertable wait時執行，不是任意搶進所有user指令。Sleep與SleepEx(alertable=TRUE)的交付條件不同；本章不把所有特殊kernel APC／thread啟動路徑概括為相同規則。

例ISR排DPC後return，DPC再排worker；若在排worker前把request context free，worker將讀失效記憶體。正確順序保留reference→提交worker→worker完成並釋放最後reference。IRQL與ownership共同限制合法動作。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

object manager：handle值只在指定table有意義

process A的handle0x40→objectF／grantedR；process B的0x40可能指eventE，不能把整數直接當跨process通用能力。正式sharing可經inherit、DuplicateHandle或以名稱再open，各自有授權／生命期規則。

步	A handle table	B handle table	object F
初始	0x40→F,R	空	有A引用
Duplicate SAME_ACCESS成功	仍0x40→F,R	0x88→F,R	兩handle同object
A close	空	仍0x88→F,R	仍不能free
B close但kernel I/O仍持reference	空	空	仍不能free
最後internal reference釋放	空	空	依物件規則可清理

SAME_ACCESS明定不增權；一般DuplicateHandle的desired-access行為與object type／呼叫權限另查API，不把所有dup都說成單純subset運算。source／target process handles須有PROCESS_DUP_HANDLE等所需權限。DuplicateHandle。

token、security descriptor與ordered DACL

token帶SID／groups／privileges等；有impersonation時還要選正確有效token。object security descriptor含owner、DACL／SACL等，DACL管允許／拒絕，SACL偏向稽核而非同一允許表。

以下簡化為enabled SIDs、普通allow/deny ACEs及已展開的specific rights，排除privilege、integrity、owner特例／object ACE；抽象R=1、W=2不是Win32實際mask。requested set=R|W，remaining初為3：

ACE順序	匹配？	remaining／結果
ALLOW Alice R	是	移除R，剩W
DENY Staff W	Alice屬Staff	W仍未授予，拒絕並停止
ALLOW Staff RW	不再執行	不能救回前段拒絕

順序有意義；若把最後ALLOW移到deny前，可先授完remaining而停止，不能無視順序稱「deny永遠蓋過一切」。系統有建議的canonical ACE排序。NULL DACL與empty DACL也不同：前者不以DACL限制，後者無entry可授權，本簡化一般請求會拒。AccessCheck。

VA保留、commit承諾及實際駐留分開

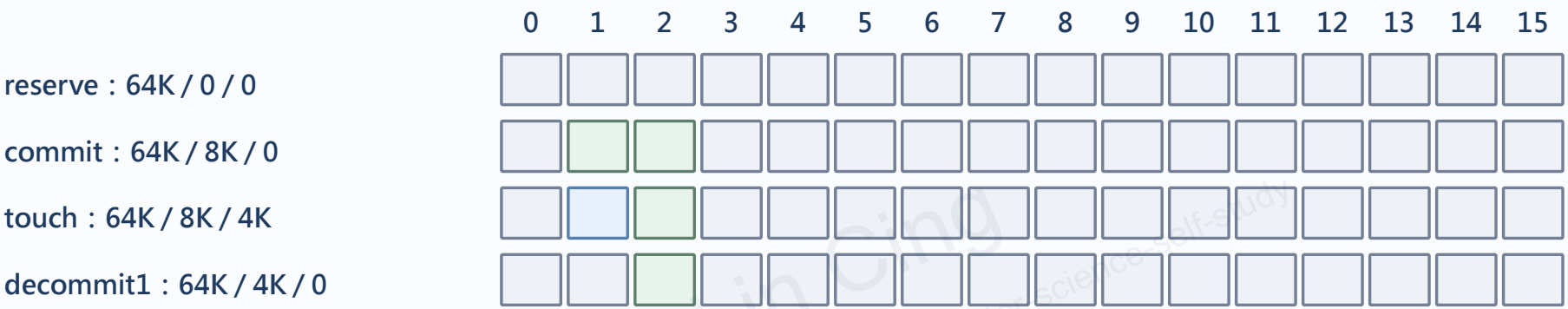
固定page4KiB、allocation granularity64KiB，一個private region基址0x100000，先reserve64KiB。reserved只占此process的VA區域；commit其中0x101000與0x102000兩頁，charge8KiB，尚不表示當下已分到兩個physical frames。

步	reserved VA	committed private	本例實際resident
reserve	64KiB	0	0
commit兩頁	64KiB	8KiB	0
首次觸及0x101123	64KiB	8KiB	4KiB，zero-fill頁
decommit第一頁且無其他引用	64KiB	4KiB	0
release整個region	0	0	0

VA保留、commit承諾及實際駐留分開 （續）

64KiB region的16個4KiB pages：reserve、commit、resident分開

灰=reserved未commit；綠=committed未resident；藍=本例resident（亦已commit）。



各列數字依序是reserved VA / committed private / resident；release整個region後三者皆0。

reserve/commit可失敗；commit受系統可承諾資源限制，不是把page file每頁立刻寫零。virtual memory也不是各process搶同一組VA數字。VirtualAlloc。

PTE、PFN與working-set trimming

PTE描述virtual page mapping／軟體狀態；PFN database描述physical frame、references與list狀態，兩者不是同一表。合法但未resident頁可能是demand-zero、file/section-backed、page-file-backed或可由RAM既有頁恢復，不把所有fault都當disk I/O。

固定VA0x401123=VPN0x401＋offset0x123；若PTE valid指PFN0x2A，PA0x2A123。若write遭RO限制且沒有合法COW條件，是access violation；不能任意補頁來繞過權限。

working set頁被trim不代表直接erase：clean且有可恢復backing者可到standby；modified需安排writeback才可供他人重用；仍pinned／I/O使用中的frame不能僅因age大就free。被trim頁若仍在standby可soft fault找回；backing不在RAM才可能要storage I/O。

Windows7用近似參考歷史／工作集heuristics管理，不把它說成知道未來的OPT或每次精確LRU。可用frame、page-table state與pending writeback要同步追蹤。

process、section與ALPC的authority

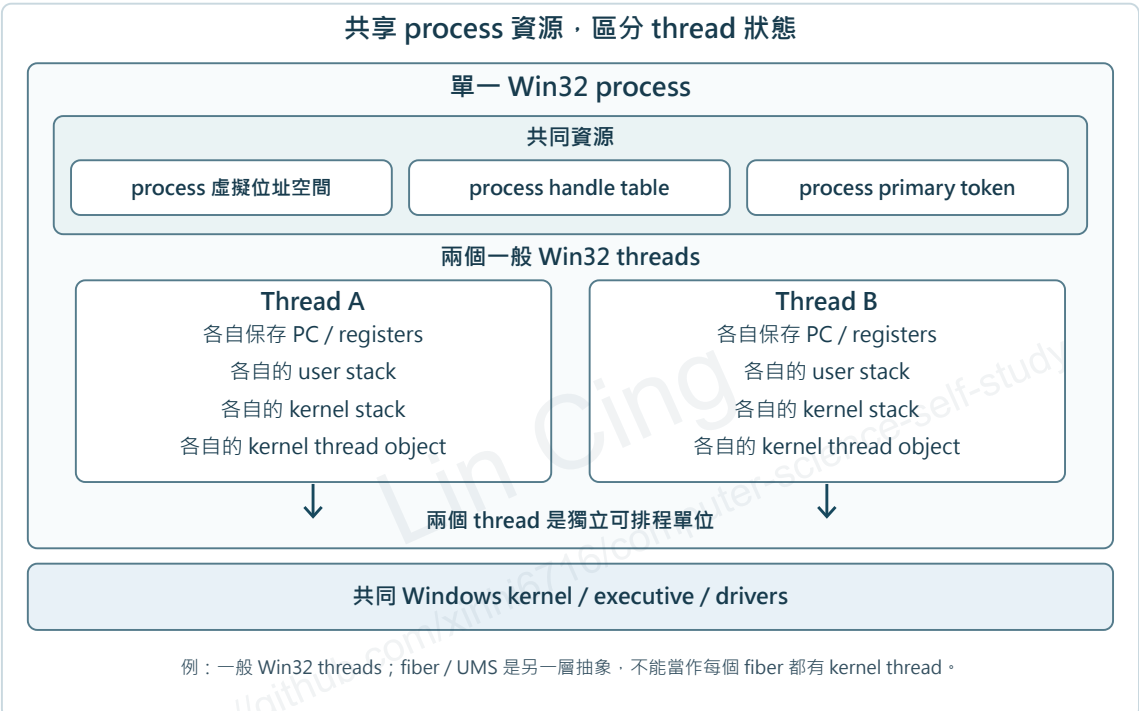
CreateProcess建立process環境並含initial thread；再CreateThread一次才是兩thread。process持address space、handle table與token等；thread有registers、user/kernel stack與執行狀態。關一個process handle不等於TerminateProcess；threadexit後object仍可供有handle者查詢／等待。

section object支援file-backed／page-file-backed映射；不同process可map不同VA而共享同一backing範圍，需各自rights及可變資料同步。ALPC／port式IPC支援client/server請求與authority傳遞；shared section可減少資料copy，但不免除buffer所有權與長度檢查。

例A VA0x100000、B VA0x500000都map section offset0的page，寫A+16可能使B+16見同一共享資料；但不把字面pointer0x100010直接交B就當有效pointer，應傳section-relative offset16和長度。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

同一 process 的兩條 Win32 threads



一般 Win32 thread 各有核心可排程的 thread object 與執行狀態；同一 process 的 threads 共用位址空間和 handle table，但各有暫存器、user stack 與 kernel stack。圖中 token 是 process 的 primary token；thread impersonation 時還須考慮其有效身分。

同一 process 的兩條 Win32 threads （續）

Fiber 和 UMS 是其他使用者層排程抽象，不能把每個 fiber 說成一條 kernel thread；共用資源也需要同步。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

IRP的向下dispatch與向上completion

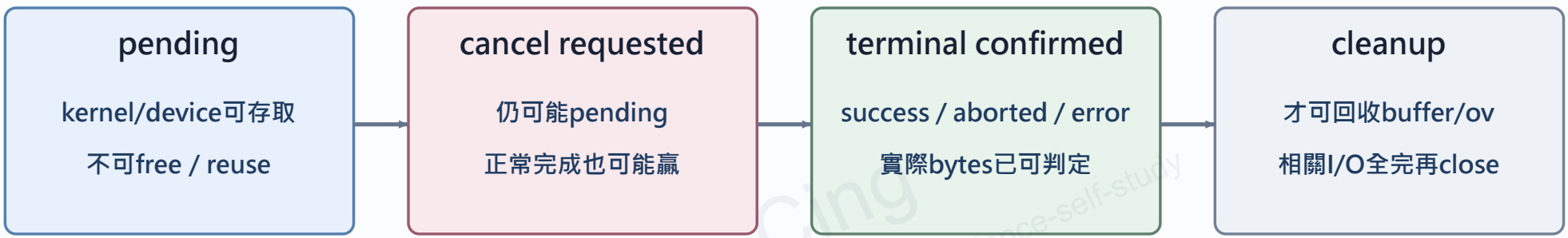
I/O manager可建立IRP，經filesystem/filter/device stack向下送；各device object有對應driver責任與per-stack狀態。底層完成時status／實際bytes沿completion路徑返回；有fast／cached路徑不必每次完全走相同IRP鏈。

時點	狀態／ownership	允許的動作
submit	caller提供buffer與唯一operation狀態	不得在pending時重用
pending	driver／device仍可能存取	取消是請求，不是立即free
hardware完成／cancel生效	status與實際bytes確定	沿completion規則發通知
caller確認terminal completion	知道success／aborted／othererror	才可回收buffer／OVERLAPPED

CancelIoEx可與正常完成競爭；其成功只表示取消已提出，ERROR_NOT_FOUND亦可能只是已無匹配pending。仍須收集原operation結果，不能只看取消回傳就結束生命期。取消規則。

IRP的向下dispatch與向上completion （續）

取消已提出 ≠ I/O已完成：buffer / OVERLAPPED須保留到terminal



圖為有提出取消的路徑；pending也可直接正常完成，取消成功回傳不代替原operation完成結果。

每個未完成operation有唯一ov / buffer；Win7 ReadFile需有效byte-count參數，最終以completion bytes為準。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

cache manager、memory manager與檔案讀取

cached read先在filesystem/cache映射找file範圍；已resident可copy到caller。若cache view缺頁，VM manager安排paging read，經儲存路徑填入frame，再滿足原read。cached I/O與paging I/O是相關而不同的request，不把一次cache miss重複當作兩份交給應用的資料。

固定讀4096B：cache hit需caller copy4096B；miss還需device→cache frame4096B（本例DMA）、最後caller copy4096B。memory copy成本、DMA bytes與storage讀取次數各有分母，不能都稱成「兩次CPU copy」。

lazy write允許dirty cache晚些寫回，改善合批但帶來durability時間差。log／data flush的次序由filesystem與storage契約建立；cache hit／write成功不自動代表斷電安全。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

PnP、power、registry與boot的依賴

PnP協調device discovery／driver stack及移除，power manager協調狀態轉換；它們都要排除仍會碰硬體的pending request後再釋放相關資源。driver object、device object與實際硬體存在／可用性是不同生命期。

registry以hives／keys／values保存配置，也可能有volatile狀態；它不是一般file pathname namespace。更改配置何時對driver生效，依服務重讀／重啟或通知契約，不是寫key就保證所有process立即換設定。

Win7 boot分firmware/boot manager／OS loader、載kernel/HAL與boot drivers、kernel初始化、session/services及logon等階段；BIOS與UEFI路徑不同，不固定全部平台都經同一MBR檔案序列。載入page tables與儲存driver等相依資源必須先可用，才能讓後面的file／memory服務工作。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

session、window station、desktop與process不是同一層

terminal services／fast user switching讓不同互動使用者保留各自session、桌面／輸入視圖及process。Windows7中service與一般互動session的安排必須與舊版混用假設分開；session隔離不把每個使用者變成獨立VM。

資源	不同sessions是否一定獨立？	理由
user address space	不同process各自有映射	不是因sessionnumber本身代替MMU
window station／desktop與部分named objects	有session相關scope	仍需object rights與指定namespace
kernel、drivers、physical RAM	共享	共同故障／資源瓶頸仍存在
NTFS files／network服務	可共享	取決於credentials與object policy

兩人同時登入並不表示本機產品允許無限制同時互動連線；功能／授權／部署限制不由kernel抽象表推論。

Local與Global名稱：scope不等於授權

固定S1/S2各建立Local\Ready event，兩者可對不同session-local objects。若依法取得Global\Ready，則可指同一global物件，但仍要通過其DACL與建立／開啟所需權利，不能以Global字樣略過access check。

handle值0x80在S1 process與S2 process可能完全不同物件；即使物件名相同也須先確定namespace與授權。session id不是可信user身分替代品。

切換前台使用者不會自動終止另一session所有process；也不能保證其持有的file鎖／CPU／memory消失。資源管理與隔離的契約應各自追蹤。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

NTFS：volume、MFT、attribute與extent

Windows7可支援不同filesystem，本章聚焦NTFS，不宣稱所有local／removable volumes必是NTFS。allocation以cluster為單位，LCN從volume起點計；若volume在磁碟partition內，轉disk byte offset須再加volume起點。

MFT保存file records／attributes，小attribute可resident在record內，大data可nonresident，透過VCN→LCN extents定位。非resident不是「沒有資料在RAM」，而是attribute內容不直接放在該MFT record。

假設record1024B、其他headers／attributes已占700，剩324；300B data在此固定預算可resident，512B放不下。真門檻依attributes／名稱／格式等，不能把300當全NTFS固定cutoff。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

VCN到LCN：完整同一檔案數例

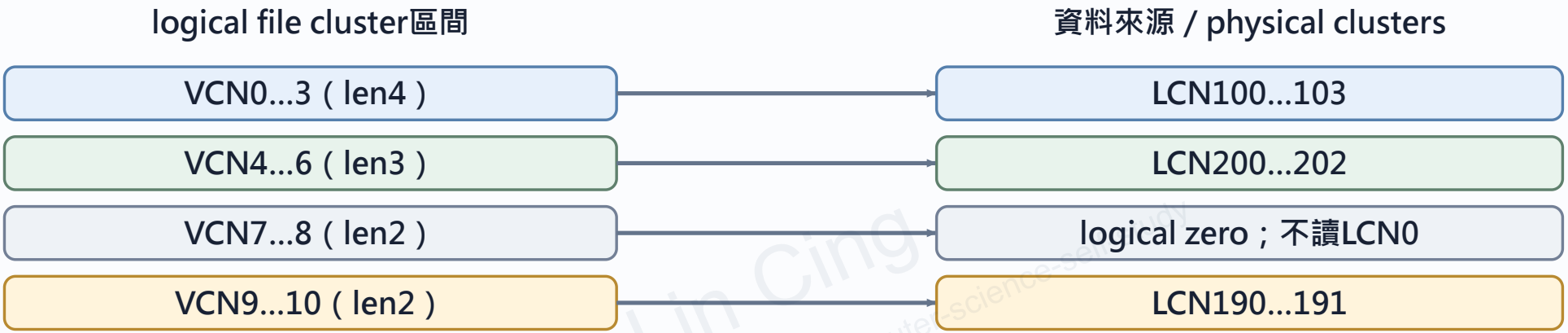
cluster4096B，volume起點disk byte1048576。file extents如下，區間均半開，None表示sparse hole：

VCN起點	長度	LCN起點	對應
0	4	100	VCN0…3 → LCN100…103
4	3	200	VCN4…6 → LCN200…202
7	2	None	VCN7/8讀出logical zeros，無data clusters
9	2	190	VCN9/10 → LCN190/191

file byte20603=5×4096+123，命中第二extent；LCN=200+(5−4)=201，volume byte=201×4096+123=823419，disk byte=1048576+823419=1871995。若落hole只填零，本例不把None算成LCN0。

VCN到LCN：完整同一檔案數例 （續）

同一NTFS file：VCN相對檔案，LCN相對volume；灰=hole



20603 → VCN5/123 → LCN201 → volume823419 → disk1871995；cluster4096B，volume基址1048576。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

file reference、目錄index與metadata files

本書NTFS file reference用48-bit MFT record number＋16-bit sequence，reuse時sequence變更，可偵測序號不符的stale reference。有限16bits不能被說成永不wrap的數學唯一身分；長期生命期仍須按實際系統規則處理。

目錄index以排序key及file reference定位；B+類索引的balanced path支持對數高度，不表示insert/split完全沒有重組成本。小index可在record／root中，長大需額外index allocation。列目錄cache的size/time欄不等於每次都已重新查最新file內容。

MFT、bitmap、log、volume與其他metadata以特殊files表達，有利共用cache／I/O機制。bitmap表示clusters的配置；其更新仍需與extent／size一致，不能各自成功就當整個allocation已atomic。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

metadata WAL：先寫log，再允許home持久化

以下為單一 transaction、無其他交易競爭且新 block 被獨占保留的 undo／redo 教學模型，不代表完整 NTFS log 格式或並行 recovery。交易所有 before／after records 和完整性標記必須**全部 durable**後，才允許任何本交易的 home 欄位更新。完整標記與 commit 可辨識，flush 可信；steal 允許 commit 前寫 home，no-force 允許 commit 後 home 尚未全寫。

舊 metadata 為 size=4096、extents=[100]、bitmap[200]=0；新狀態為 size=8192、extents=[100,200]、bitmap[200]=1。完整 log 先 durable，再可更新 home；commit durable 後才宣布交易成功。

crash 時紀錄	home 可能狀態	recovery
完整 log 尚未 durable	全部 old	此模型禁止任何 home 更新
完整 log durable、無 durable commit	old 或混合	用 before-images undo 到完整 old
完整 log 與 commit 均 durable	old、混合或 new	用 after-images redo 到完整 new

恢復選擇整組 metadata。實際 NTFS 還處理 LSN、並行與 checkpoint 等。

恢復不變量、checkpoint及user data範圍

在上述模型，任何 home 更新之前，全交易 before／after log 與完整性標記均已 durable。無 durable commit 時，用 before-images undo 已更動欄位；有 durable commit 時，用 after-images redo 所有新值。以 assign old／new 重播具冪等性；log 須保留，且故障與 flush 符合模型假設。

checkpoint 不表示所有較早的 log 均可刪：活躍 transaction 的 undo、尚未落 home 的 dirty metadata 的 redo 須保留到安全截斷條件成立。

metadata 一致不保證 user file 內容構成完整應用 transaction。write 成功可能只在 cache；兩個業務檔也不因 NTFS 有 log 而自動跨檔原子。應用仍須設計 flush、發布或交易協定。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

ACL、EFS、BitLocker與repair處理不同威脅

NTFS連到Windows security descriptor／DACL模型；不是把POSIX owner/group/other mask算法套進來。平常有bypass-traverse privilege的情境與逐directory FILE_TRAVERSE檢查也不同，不能一概宣稱Windows不做路徑保護。

EFS在file內容層加密，BitLocker在volume／儲存保護層；它們各自有key與解鎖條件，不能單靠NTFS ACL保護被搬到另一系統直接讀取的未加密媒體。加密不自動提供backup或應用transaction。

bad-sector remapping更換位置；若舊data已無法讀又沒有有效mirror/parity等來源，重新配好cluster也不會創造原內容。RAID／spares、metadata recovery與錯誤資料修復是不同保證。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

sparse、compression、USN與snapshot分開追蹤

前述file共11logical clusters，其中2holes，非hole data配置9clusters，若不另compression則36864B；logical size可為45056B。hole讀零，將來寫入時另需allocation，不是把整個file都先保留9clusters大小的VA。

NTFS compression以compression units等規則表達，效果依內容及配置條件；不能宣稱任意file都固定壓縮比。USN change journal記錄可供追蹤／索引的變更，與metadata recovery log不同；journal可wrap，consumer遇歷史gap需重新建立狀態，不能視為永遠完整的backup。

VSS snapshot給一個時間點的storage視圖；若應用當時正更新多個相關欄位，仍可能需要writer協調／quiescence才有application-consistent狀態。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

snapshot保留的是舊版，不是把新版誤當備份

copy-on-write snapshot簡化模型：live blocks X=41、Y=80，建立snapshot S。首次把live X改42前，先讓S保留old X41，再允許live指新X42；後續live X43，S仍讀41，不再改成42。Y沒被改，S/live可共用同old backing80。

時點	live X	S X	Y
snapshot後	41	41	live/S共用80
第一次寫後	42	41	仍80
第二次寫後	43	41	仍80

old backing只有在不再被任何snapshot／reader需要時可回收。snapshot若與live共用同一故障儲存，單次device全失仍會一起失去；不是單獨備份媒體。

networking interfaces與protocol責任

user Winsock提供socket概念；NDIS處理網路adapter／protocol相接的driver模型。原書TDI屬歷史介面討論；官方對Vista及以後新kernel network modules引導使用WSK，不把TDI當Windows7新driver唯一途徑。這些是API／driver分工，不能直接對應為每個packet必經幾次process switch。

機制	處理什麼	不直接保證
TCP/IP	位址／封包／可靠byte stream等層級	應用訊息邊界、業務exactly-once
SMB redirector/server	remote file operations	localfile與remote failure完全相同
named pipe	本機／遠端pipe語意	byte與message mode可互換
RPC	把請求／回覆包成遠端呼叫	timeout表示server未執行
COM	物件介面與marshalling等	每個COMcall都跨機器

協定版本、認證／加密與錯誤語意要另外指定。HTTP/WebDAV、VPN等名稱也不能代替安全設定；歷史介紹不作當代安全建議。

remote file：UNC命名不抹掉partial failure

例應用開`\\server\share\report`，I/O路徑辨識remote名稱，redirector將操作編碼成network request，server經自己的filesystem及credentials政策處理，再回status／bytes。client kernel、network、server與server storage可各自失效。

client request送達，server寫入後reply遺失：client timeout不能推出「沒寫」。若重試非冪等append，可重複新增；需要協定／server request identity、重試規則與持久化語意。TCP可靠stream本身不解業務重複執行。

redirector使API可相似，但不同share／offline cache策略會改visibility及錯誤時點。成功open也不保證連線之後永不失效。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

named pipe模式、domain與directory services

固定為 message-type named pipe，且本次接收 handle 使用 `PIPE_READMODE_MESSAGE`。若訊息為 500 B、buffer 容量 200 B，本例可取得 200 B 與 `ERROR_MORE_DATA`，再讀同訊息餘下的 300 B；FALSE 不一律代表零 bytes 的致命錯誤。Message-type pipe 的 handle 亦可使用 byte-read mode，不能把本例套用到該模式或 TCP。

domain 以共同 identity／管理協定整合資源；Active Directory 結合 directory 與認證服務，不只是「加密資料庫」。相同顯示名稱不代表相同 SID，server 應依有效 credentials、token 與 object policy 判定。

client-side caching 可供離線工作，但重新連線有版本及衝突問題；cached success 不自動等於 remote durable commit。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

CreateProcess、threads、fibers與UMS的排程單位

CreateProcess成功包含一個initial thread；process object與thread object分別可由handle觀察／等待。再建第二thread時可共享address space與某些process資源，但各自保存PC、stack、TLS等。可用句柄名稱不能替代reference lifetime。

fibers由user code在thread內切換；若承載thread執行blocking system call，普通fiber切換本身不會讓這個kernel thread同時替另一fiber執行。Windows7 UMS是特定平台／API條件下讓user scheduler與可管理的kernel執行狀態配合，不能簡化成所有阻塞都免費或所有Win32 API自動相容任意fiber。

thread pool以重用workers／queue工作減少反覆create成本，仍需管理工作資料生命期。單worker pool的job A等待同pool內尚未執行的job B，可造成pool starvation deadlock。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

handle sharing與authority不能只傳一個整數

inherit需要來源handle的inherit設定與CreateProcess的相應條件／指定handle list；DuplicateHandle需來源與目標process的合法操作權限；named open則在指定namespace重新依security policy取得handle。

明定DUPLICATE_SAME_ACCESS時，新handle保持原granted access；A有R但無W，不因複製給child就變W。若要取得不同access，必須按object type及API授權規則處理，不假設可任意增權，也不把本例subset模型冒稱所有DuplicateHandle行為。

對同一file object的duplicated handles可共享current file position；獨立CreateFile可得到不同file position。後續同步seek/read仍可能競爭，overlapped I/O用每個operation自己的offset與狀態。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

ReadFile三種起始結果與Win7參數

採regular file且handle用FILE_FLAG_OVERLAPPED開啟，每個未完成operation使用獨立OVERLAPPED與有效buffer；本例用manual-reset event做完成通知。Windows7要求ReadFile的lpNumberOfBytesRead非NULL，提供DWORD位置，但pending時以最後completion回報的bytes為準。

ReadFile起始結果	意思	生命期
TRUE	這次已完成，檢查實際bytes	可按完成結果處理
FALSE＋ERROR_IO_PENDING	已提交且尚未完成，不是一般失敗	buffer/OVERLAPPED/event保持有效
FALSE＋其他error	依file／pipe／device契約處理	先確認是否仍有未完成operation

pending可用GetOverlappedResult等相應完成機制確認。不要拿現代GetOverlappedResultEx範例回填為Win7必有；本章例程使用較早的GetOverlappedResult。

同一file的6、4、EOF追蹤

固定 regular file 為 ABCDEFGHIJ，共 10 B；沒有並行寫入或 truncate。每次用 `FILE_FLAG_OVERLAPPED` handle 請求 6 B，並指定獨立有效的 OVERLAPPED、buffer 與 offset。R0、R1 均成功；起始呼叫可立即完成或先返回 pending。

operation	offset	request	最終結果	下一步
R0	0	6	成功 6 B：ABCDEF	terminal 後才重用狀態
R1	6	6	成功 4 B：GHIJ	已讀前綴共 10 B
R2	10	6	EOF：`ERROR_HANDLE_EOF`	停止

本例 overlapped 正長度讀取，R2 可由 `ReadFile` 立即回報 `ERROR_HANDLE_EOF`，或先 pending 後由 `GetOverlappedResult` 回報；立即失敗時沒有 pending 工作可等待。另一種非 overlapped 同步讀取到 EOF 可成功返回 0 B，不能混進 R2 的契約。pipe／device 有不同規則；取消可能輸給正常完成，須收集實際結果。

memory、heap、mapping與TLS的不同層級

VirtualAlloc管理VA reserve／commit及page protection；heap manager在其取得的區域內細分小物件，HeapFree不是自動release整個region。file mapping／section為file-backed或page-file-backed view，MapView取得各process內的VA；視圖大小／offset與實際file大小需一致。

TLS提供per-thread slot，保護的是索引／邏輯歸屬，不阻止同address space另一thread若有pointer就存取該記憶體。物件超出thread生命期時需明確owner／同步；TLS不等於獨立MMU頁表。

virtual commit、working set與heap allocation各有不同統計量。64KiB reserve、8KiB commit、4KiB resident可同時成立；不能拿一個數說明全部memory使用。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

取消與cleanup的完整協定

本章的安全不變量：只要operation仍可能被kernel／device碰觸，buffer、OVERLAPPED、event及其context就必須存活。

對已pending R：CancelIoEx(h,&ov)提出取消→用原完成管道確認R terminal→處理normal／aborted／othererror與實際bytes→釋放資料／ov，最後在所有相關operations終止後關handle與event。

如果取消輸給正常完成，R可success；如果取消成功中止，R可ERROR_OPERATION_ABORTED；取消回ERROR_NOT_FOUND也不取代原R結果。不得在取消請求回來後立刻free，亦不得同一ov同時承載R1/R2。不同completion API／IOCP通知模式有自己的交付規則，需與所選模型一致。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

同一個read的各層狀態

層	需要追蹤	可能已成功但後面仍失敗的情況
authority	process-local handle、granted mask、有效token	有handle不代表有W或永久network可用
virtual memory	buffer VA、commit、resident、page rights	reserve成功但尚不可觸及未commit頁
I/O	IRP／operation state、pending、actual bytes	提交成功不代表完成
cache／filesystem	file offset、VCN/LCN、metadata一致性	write cache完成不代表user data durable
device／network	DMA ownership、連線、storage contract	取消已提出不代表硬體已停止

IRQL限制目前可做的kernel操作；thread priority決定何者在一般排程中先取得CPU。混為同一priority會讓ISR/DPC生命期與等待推理一起出錯。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

本章應能重建的四個例子

1. 同priority q2且P3於t1以priority10到達：P1/P2/P3完成8/9/3；每次選取回到queue與剩餘slice。
2. region64KiB reserve／8KiBcommit／4KiBresident；decommit與release各改不同集合。
3. NTFS file byte20603→VCN5/123→LCN201→volume823419→disk1871995；hole不當成LCN0。
4. ReadFile offsets0/6/10，完成6/4/EOF；pending、cancel-request、terminal-completion分開，才有合法cleanup。

這些例子連到一般不變量：grant只允許持有right、已發生I/O引用不得早free、file配置與bitmap／size須一致、shared memory及其他processors不受單CPU IRQL自動互斥。

參考資料與署名

Lin Cing，《作業系統》手寫筆記。 Silberschatz et al., Operating System Concepts, 9th ed., Ch. 19. Microsoft: IRQL, VirtualAlloc, AccessCheck. ReadFile, EOF, CancelIoEx, named pipes.

Lin Cing 原創教學整理，原創內容以 CC BY 4.0 授權。

https://github.com/xinhi6716/computer-science-self-study